



Dekoratory w języku Python i metaprogramowanie

Jacob Czajka, Maciej Miłek

CZĘŚĆ I

Dekoratory jako wzorzec projektowy i ich
implementacja

DEFINICJA I CECHY DEKORATORA

Definicja: Dekorator to funkcja (lub klasa), która przyjmuje inną funkcję jako argument i zwraca jej nową wersję, wzbogaconą o dodatkową logikę, bez modyfikowania kodu źródłowego dekorowanego obiektu.

🔗 Separacja logiki

Oddziela główne zadanie funkcji od zadań pobocznych (np. logowania, autoryzacji).

✍ Syntactic Sugar

Użycie symbolu @ czyni kod czytelny i eleganckim.

≡ Dynamiczność

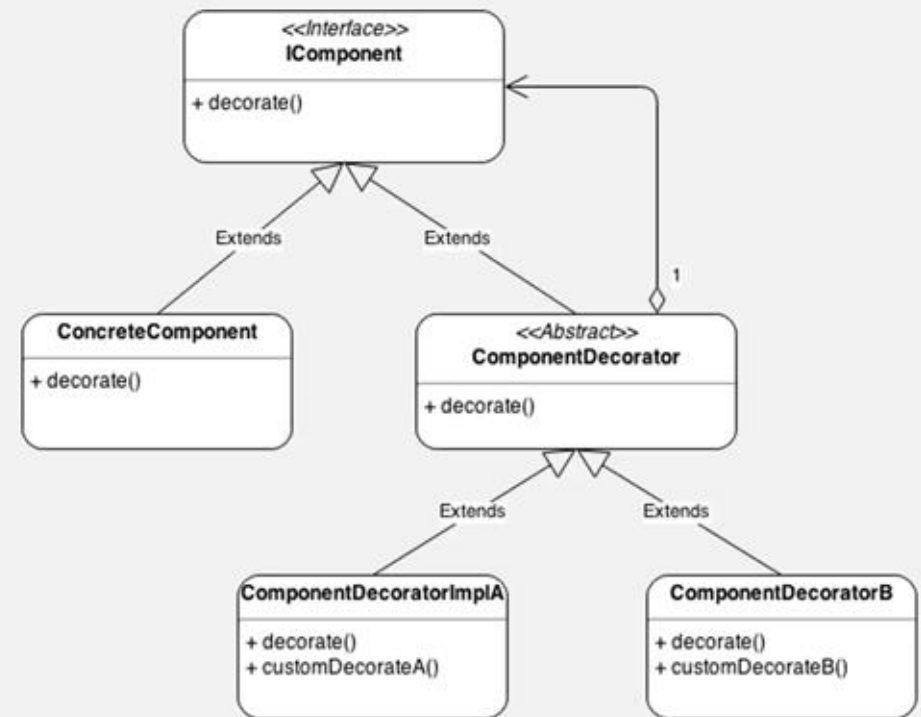
Umożliwia modyfikację zachowania w czasie runtime.

STRUKTURA WZORCA

```
def dekorator_powitania(funkcja_oryginalna):  
    def opakowanie(uzytkownik):  
        print("Logowanie: sprawdzanie uprawnień")  
        wynik = funkcja_oryginalna(uzytkownik)  
        return wynik  
    return opakowanie  
  
@dekorator_powitania  
def wyswietl_wiadomosc(uzytkownik):  
    print("Witaj w systemie, " + uzytkownik)
```

Mechanizm oparty na funkcjach domknięcia (closures) i funkcjach wyższego rzędu.

Decorator pattern – Class diagram



KONSEKWENCJE STOSOWANIA

+Zalety

- Reużywalność kodu (DRY).
- Zasada Otwarte-Zamknięte (SOLID).
- Uproszczenie logiki biznesowej.

-Wyzwania

- Trudniejsze debugowanie.
- Zamaskowanie metadanych (wymaga wraps).
- Koszt wydajnościowy (dodatkowe wywołania).

PRZYKŁADOWE ZASTOSOWANIA

```
import time

def timer(funkcja_do_pomiaru):
    def opakowanie(liczba):
        start = time.perf_counter()
        wynik = funkcja_do_pomiaru(liczba)
        koniec = time.perf_counter()
        print("Czas wykonania: ", koniec - start)
        return wynik
    return opakowanie
```

Dekorator `timer` służy do profilowania wydajności poprzez automatyczny pomiar czasu wykonania operacji dla zadanej liczby.

```
from functools import lru_cache

@lru_cache(maxsize=128)
def oblicz_silnie(n):
    if n < 2:
        return 1
    return n * oblicz_silnie(n - 1)
```

Wykorzystanie dekoratora `lru_cache` do memoizacji, co zapobiega wielokrotnemu obliczaniu tych samych wartości silni.

CZĘŚĆ II

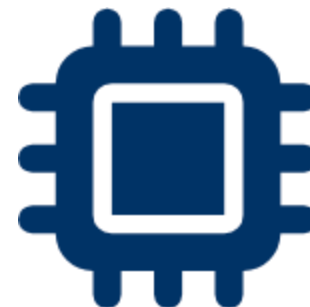
Metaprogramowanie: Introspekcja, Refleksja i
Metaklasy

DEFINICJA METAPROGRAMOWANIA

Metaprogramowanie to tworzenie programów komputerowych, które traktują inne programy jako swoje dane. Oznacza to, że program może czytać, generować, analizować lub modyfikować inne programy, a nawet samego siebie, podczas działania.

Główne cechy:

- Wysoka abstrakcja kodu.
- Automatyczne generowanie struktur.
- Adaptacja zachowania w runtime.
- Zmniejszenie ilości powtarzalnego kodu (boilerplate).



"Kod piszący kod"

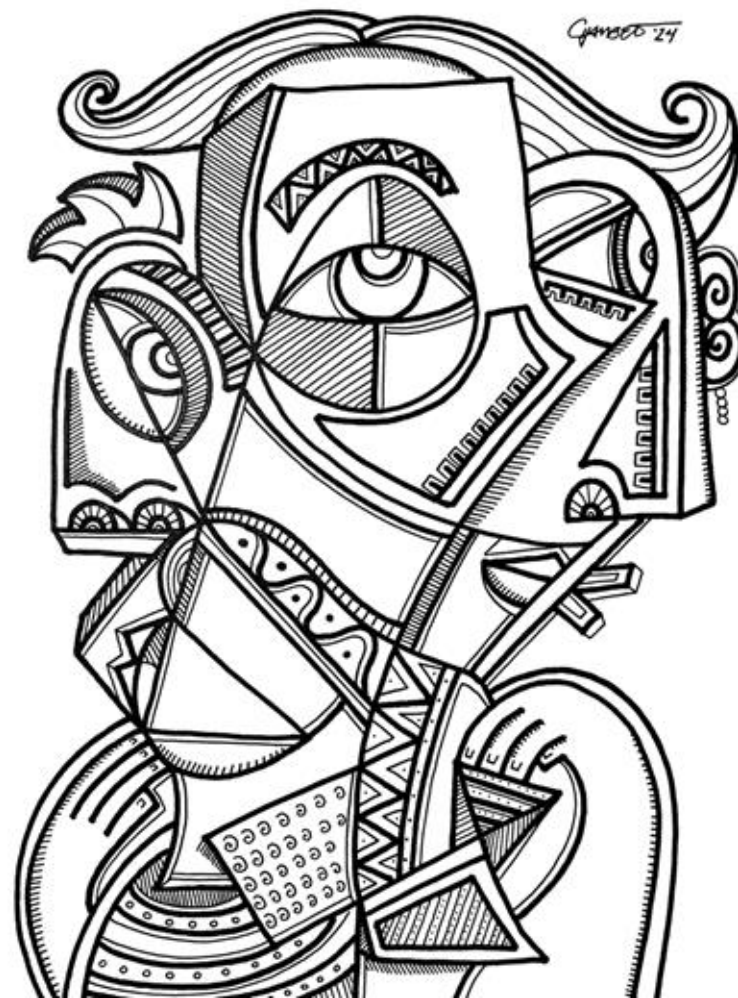
INTROSPEKCJA I REFLEKSJA

QIntrospekcja

Zdolność programu do badania typu i właściwości obiektu, np. `type()`, `dir()`, `hasattr()`.

Refleksja

Zdolność do modyfikowania własnej struktury, np. `setattr()`, dynamiczne dodawanie metod do klasy.



MAGICZNE METODY (DUNDER)

Definicja: Metody magiczne (double underscore) to specjalne haki (hooks) wbudowane w Pythona, wywoływane niejawnie przez interpreter w celu realizacji podstawowych operacji na obiektach.

Metoda	Operacja	Przykład
<code>__init__</code>	Inicjalizacja obiektu	<code>obj = Klasa()</code>
<code>__call__</code>	Wywołanie obiektu jako funkcji	<code>obj()</code>
<code>__getitem__</code>	Dostęp przez indeks/klucz	<code>obj['klucz']</code>
<code>__new__</code>	Tworzenie nowej instancji	Kluczowe dla metaklas

MAKRA LISPOWE

Definicja: Makra w języku Lisp to funkcje przekształcające drzewo składniowe (AST) przed jego ewaluacją. W przeciwieństwie do funkcji w Pythonie, makra operują na "kodzie jako danych" (homoikoniczność).

```
(defmacro when (condition &rest body)
  `(if ,condition
      (progn ,@body)))
```

Makra pozwalają na definiowanie własnych struktur sterujących.



PRZYKŁADY METAPROGRAMÓW W PYTHONIE

1. Dynamiczne klasy

```
Uzytkownik = type(
    'Uzytkownik',
    (object,),
    {'wiek': 25, 'status': 'aktywny'}
)
print("Utworzono klase dla obiektu uzytkownik")
```

W Pythonie klasy są obiektami, które można tworzyć dynamicznie w czasie działania programu (runtime). Zamiast standardowego słowa kluczowego `class`, używamy funkcji `type()` z trzema argumentami: nazwą klasy, krotką klas bazowych (dziedziczenie) oraz słownikiem atrybutów.

2. Prosta metaklasa

```
class MetaWalidacja(type):
    def __new__(metaklasa, nazwa, bazy, atrybuty):
        print("Analizowanie klasy: ", nazwa)
        return super().__new__(
            metaklasa, nazwa, bazy, atrybuty
        )
```

Metaklasa to „klasa klas” – definiuje ona zachowanie i sposób powstawania innych klas. Przez nadpisanie metody `__new__` w klasie dziedziczącej po `type`, możemy „wstrzyknąć” własną logikę w momencie, gdy interpreter Pythona napotyka definicję nowej klasy. W przykładzie kod wypisuje nazwę analizowanej klasy, zanim zostanie ona ostatecznie utworzona.

BIBLIOGRAFIA

Literatura:

- Gamma E. et al., "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley
- Ramalho L., "Fluent Python: Clear, Concise, and Effective Programming", O'Reilly Media.
- Beazley D., Jones B.K., "Python Cookbook", O'Reilly Media.
- Python Software Foundation, "Python Documentation: Data Model & Metaclasses".
- Refactoring.Guru, "Decorator Pattern in Python".
- Graham P., "On Lisp: Advanced Techniques for Common Lisp", Prentice Hall.

Dziękujemy za uwagę!

Źródła obrazów



<https://4kwallpapers.com/images/wallpapers/python-programming-2732x2732-16046.jpg>



https://reactiveprogramming.io/_next/image?url=%2Fbooks%2Fpatterns%2Fimg%2Fpatterns-articles%2Fdecorator-diagram.png&w=3840&q=75



<https://images.squarespace-cdn.com/content/v1/60985832b60ac56721f20048/1727302508981-5QDWTGFMVPNUITE694MO/photo-output.png>



https://upload.wikimedia.org/wikipedia/commons/4/48/Lisp_logo.svg